

Tellisi App: Arabic Sign Language Letter Recognition Using Fine-tuned MobileNetV2 and Flutter-based Mobile Deployment

Mohamed Al-Tellisi

Information Technology (IT)
Department of Electrical and Computer Engineering
School of Applied Sciences and Engineering
Libyan Academy for Postgraduate Studies
Email: 250101283@academy.edu.ly

Supervised by Dr. Abdulmawla Al-Najeh

Department of Electrical and Computer Engineering
School of Applied Sciences and Engineering
Libyan Academy for Postgraduate Studies

Abstract—Arabic Sign Language recognition is an important computer vision task that can support communication accessibility for deaf and hard-of-hearing individuals. This paper presents Tellisi App, an Arabic Sign Language letter recognition system based on deep learning and mobile deployment. The study focuses on five Arabic sign language letters: Alef, Beh, Teh, Jeem, and Seen. A sequence of experiments was conducted to evaluate different learning strategies, beginning with a baseline convolutional neural network (CNN) trained from scratch, followed by an improved CNN using data augmentation and dropout, and finally a transfer learning approach based on MobileNetV2. The baseline CNN achieved 51.44% test accuracy, while the improved CNN reached 69.23%. MobileNetV2 feature extraction improved the accuracy to 88.46%, and fine-tuning the final layers of MobileNetV2 further increased the test accuracy to 95.19%. The final model was evaluated using accuracy, loss, precision, recall, F1-score, and a confusion matrix. Error analysis showed that most remaining errors occurred between visually similar signs, especially Beh and Teh. The trained model was converted to TensorFlow Lite and integrated into a Flutter mobile application that supports both image upload recognition and live camera recognition. The proposed system demonstrates a complete artificial intelligence workflow, including dataset preparation, model training, performance improvement, evaluation, error analysis, and mobile deployment.

Index Terms—Arabic Sign Language, deep learning, CNN, MobileNetV2, transfer learning, fine-tuning, TensorFlow Lite, Flutter, image classification.

I. INTRODUCTION

Sign language is an essential communication method for many deaf and hard-of-hearing individuals. However, communication barriers still exist between sign language users and people who do not understand sign language. Automatic sign language recognition can help reduce this gap by using computer vision and machine learning to interpret hand gestures and convert them into readable or audible outputs. In recent years, deep learning has become one of the most successful approaches for image classification because it can automatically learn visual features from raw images instead of relying only on manually designed features.

Arabic Sign Language recognition is particularly important because Arabic-speaking communities require accessible tools that support their language and cultural context. Although many sign language recognition systems have been developed for other languages, Arabic Sign Language remains an important area for further research and practical implementation. The availability of RGB image datasets and modern deep learning models makes it possible to build recognition systems that can classify Arabic sign language letters from images or camera input.

This project presents Tellisi App, a mobile-based Arabic Sign Language recognition system. The project uses a ready RGB Arabic alphabet sign language dataset and applies several deep learning experiments to improve classification accuracy. The work begins with a baseline CNN model trained from scratch in order to establish an initial performance level. Then, data augmentation and dropout are introduced to improve generalization. Finally, transfer learning using MobileNetV2 is applied, followed by fine-tuning to further improve performance.

A key academic point in this project is that the final model does not claim to introduce a new neural network architecture. Instead, the project demonstrates a complete artificial intelligence pipeline for a practical recognition problem. The baseline CNN was trained from scratch, while the final model used transfer learning and fine-tuning, which are established and scientifically valid methods when the available task-specific dataset is limited. This distinction is important for academic honesty and for defending the work during discussion.

The main contributions of this paper are summarized as follows:

- Building and evaluating a baseline CNN model trained from scratch for Arabic Sign Language letter classification.
- Improving the baseline model using data augmentation, dropout, and longer training.
- Applying MobileNetV2 transfer learning and fine-tuning to improve classification accuracy.

- Evaluating the final model using accuracy, loss, precision, recall, F1-score, and confusion matrix analysis.
- Performing error analysis to identify the most confusing sign classes and to guide dataset improvement.
- Deploying the final model into a Flutter mobile application using TensorFlow Lite.
- Providing two recognition modes: image upload recognition and live camera recognition.

The remainder of this paper is organized as follows. Section II reviews related work and background concepts. Section III describes the dataset and preprocessing steps. Section IV presents the proposed methodology and model architectures. Section V explains the experimental setup. Section VI discusses the results and evaluation. Section VII presents error analysis and dataset improvement strategy. Section VIII explains the mobile application integration. Section IX discusses limitations and future work. Finally, Section X concludes the paper.

II. RELATED WORK AND BACKGROUND

Sign language recognition has become an active research area within computer vision and human-computer interaction. Traditional approaches often relied on handcrafted features, such as hand shape, contour, skin color segmentation, edge descriptors, or motion features. These methods can be useful in controlled environments, but they are sensitive to lighting conditions, background variation, hand orientation, and camera quality. For practical applications, especially mobile applications, the recognition system must be robust to different capture conditions.

Deep learning, especially convolutional neural networks, has significantly improved image classification tasks because CNN models can automatically learn visual features directly from image data. LeCun et al. demonstrated the effectiveness of gradient-based learning for visual pattern recognition in early CNN research [4]. Since then, CNNs have become a standard approach for image-based classification tasks. In sign language recognition, CNN-based systems can learn patterns related to hand shape, finger positions, and visual gesture structure. However, training deep CNN models from scratch usually requires a large amount of labeled data. When the dataset is limited, the model may overfit or fail to generalize well to unseen images.

Transfer learning is commonly used to address this limitation. Instead of training a deep model entirely from scratch, a pre-trained model is used as a feature extractor or fine-tuned for a new task. ImageNet and the ImageNet Large Scale Visual Recognition Challenge played a major role in enabling large-scale pre-training for visual recognition models [3]. A model pre-trained on a large image dataset learns general visual features such as edges, textures, shapes, and higher-level object parts. These features can then be reused for specialized tasks with smaller datasets.

MobileNetV2 is especially suitable for mobile and embedded applications because it is designed to be lightweight and efficient while maintaining strong feature extraction capability.

Sandler et al. introduced MobileNetV2 using inverted residuals and linear bottlenecks, enabling efficient computation while preserving representational power [2]. In this project, MobileNetV2 is used first as a frozen feature extractor and then fine-tuned on the Arabic Sign Language letter dataset.

For mobile deployment, TensorFlow Lite provides a practical approach for converting trained TensorFlow/Keras models into lightweight models that can run on mobile devices [7]. In this project, the best-performing Keras model is converted into a TensorFlow Lite model and integrated into a Flutter application. This allows the recognition model to run locally on the device without requiring a remote server, which is useful for privacy, latency, and offline demonstration.

Arabic Sign Language datasets are essential for building recognition systems that are relevant to Arabic-speaking communities. The RGB Arabic Alphabet Sign Language dataset introduced by Al-Barham et al. contains 7,856 raw and fully labeled RGB images collected from more than 200 participants and made publicly available through Kaggle [1]. This dataset provides the foundation for the experiments in this work. In this project, a subset of five letters was selected to build a focused and well-analyzed recognition prototype.

III. DATASET DESCRIPTION

This study uses an RGB Arabic alphabet sign language dataset for image-based Arabic Sign Language letter classification. The complete dataset contains RGB images of Arabic sign language alphabet gestures [1]. In this project, five classes were selected in order to build a focused and manageable recognition system suitable for a semester-level artificial intelligence project. The selected classes are Alef, Beh, Teh, Jeem, and Seen.

The selected subset contains 1,381 images distributed across the five classes. The number of images per class is not perfectly balanced. Alef contains 287 images, Beh contains 307 images, Teh contains 311 images, Jeem contains 210 images, and Seen contains 266 images. This indicates that the dataset is moderately imbalanced, with Jeem having the lowest number of samples. The distribution is shown in Fig. 1.

The dataset was divided into training, validation, and testing subsets. A stratified splitting strategy was used to preserve the class distribution across the three subsets. The dataset was first divided into 70% training data and 30% temporary data. The temporary data was then divided equally into validation and testing subsets. This resulted in 966 training images, 207 validation images, and 208 testing images.

Stratified splitting was important because the selected dataset was not perfectly balanced. Without stratification, some classes could be overrepresented or underrepresented in one subset, which would make the evaluation less reliable. The test set was kept separate from training and validation in order to provide a final evaluation of the trained models. The train, validation, and test split is summarized in Table I.

IV. PREPROCESSING AND DATA AUGMENTATION

Before training, all images were prepared according to the input requirements of each model. For the baseline CNN

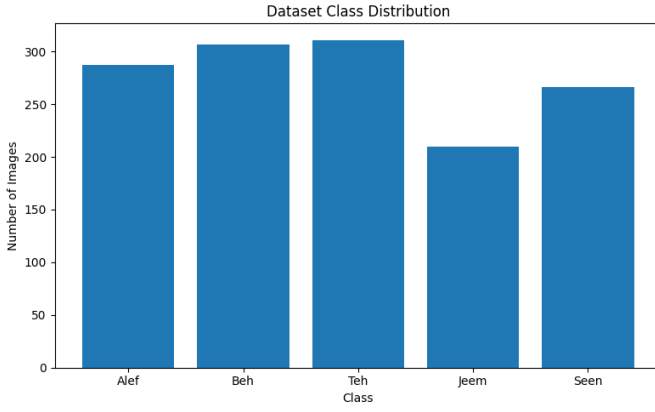


Fig. 1: Dataset class distribution for the selected Arabic Sign Language letters.

TABLE I: Dataset Split Summary

Subset	Number of Images	Percentage
Training	966	69.95%
Validation	207	14.99%
Testing	208	15.06%
Total	1,381	100%

experiments, images were resized to 64×64 pixels and normalized by scaling pixel values from the range $[0, 255]$ to $[0, 1]$. This reduced computational cost and allowed the baseline model to train quickly.

For the MobileNetV2 experiments, images were resized to 224×224 pixels, which matches the expected input size used by the MobileNetV2 architecture. The MobileNetV2 preprocessing function was applied to transform pixel values from the range $[0, 255]$ to the range $[-1, 1]$. This step was necessary to maintain consistency between the training process and the preprocessing expected by the pre-trained MobileNetV2 model. The same preprocessing operation was later implemented inside the Flutter application before TensorFlow Lite inference.

Data augmentation was applied to improve generalization and reduce overfitting. The augmentation techniques included small rotations, zooming, width shifting, height shifting, and brightness variation. These transformations were selected because real-world sign images may vary in camera angle, hand position, distance, and lighting conditions. Horizontal flipping was not used because flipping a hand sign may change its visual meaning and could introduce incorrect training samples.

The augmentation strategy was intentionally realistic. Excessive transformations can create images that no longer represent valid sign gestures, while weak augmentation may not expose the model to enough variation. Therefore, moderate augmentation values were used to simulate practical camera conditions without changing the semantic meaning of the sign.

V. PROPOSED METHODOLOGY

The proposed system follows a progressive experimental methodology. Instead of directly using a high-performing pre-trained model, the work begins with a simple baseline model trained from scratch. This provides an initial reference point for comparison. Then, the model is improved using regularization and data augmentation. Finally, transfer learning and fine-tuning are applied using MobileNetV2. The full workflow is shown in Fig. 2.

A. Baseline CNN Model

The first experiment used a simple convolutional neural network trained from scratch. The model consisted of convolutional layers, max-pooling layers, a flattening layer, and fully connected dense layers. The output layer used a softmax activation function with five output neurons, one for each selected sign class.

The baseline CNN was trained for 5 epochs using categorical cross-entropy as the loss function and Adam as the optimizer. This model was intentionally simple in order to establish a baseline performance level. The baseline model achieved 51.44% test accuracy. Although this result was not high enough for a practical system, it was useful because it provided a fair baseline for measuring later improvements.

B. CNN with Data Augmentation and Dropout

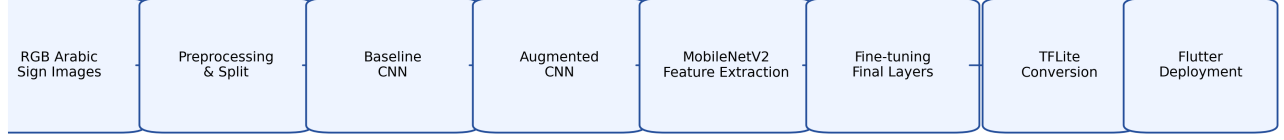
The second stage improved the baseline CNN by increasing model capacity, adding data augmentation, and applying dropout. Data augmentation allowed the model to see different variations of the same training images, while dropout reduced the risk of overfitting by randomly disabling a portion of neurons during training. This encouraged the model to learn more robust features instead of memorizing specific training examples.

This improved CNN was trained in several experiments. Training for 10 epochs achieved 53.36% test accuracy. Increasing the training duration to 20 epochs improved the test accuracy to 65.38%. Training for 40 epochs further improved the test accuracy to 69.23%. These results show that augmentation and longer training improved the baseline model, but the CNN trained from scratch still had limited performance compared to the transfer learning approach.

C. MobileNetV2 Feature Extraction

The third stage used MobileNetV2 as a pre-trained feature extractor. MobileNetV2 was selected because it is lightweight and suitable for mobile and embedded applications. The base MobileNetV2 model was initialized with ImageNet weights and used without its original classification head. The base layers were frozen, and new classification layers were added on top of the model.

The added classification head included a global average pooling layer, a dense layer with ReLU activation, a dropout layer, and a final softmax output layer with five classes. During this stage, only the newly added classification layers were trained, while the MobileNetV2 base remained frozen. This



Progressive AI workflow: train from scratch, improve, transfer learn, fine-tune, evaluate, and deploy.

Fig. 2: Proposed AI workflow used in Tellisi App. The project starts with a CNN baseline trained from scratch, improves it using augmentation, then applies transfer learning, fine-tuning, evaluation, TensorFlow Lite conversion, and Flutter deployment.

approach significantly improved the performance, achieving 88.46% test accuracy.

D. MobileNetV2 Fine-tuning

After training the classification head, fine-tuning was applied to improve the model further. In this stage, a portion of the final MobileNetV2 layers was unfrozen and trained using a small learning rate of 1×10^{-5} . Batch normalization layers were kept frozen to avoid destabilizing the learned feature distributions. Fine-tuning allowed the model to slightly adapt the higher-level visual features of MobileNetV2 to the Arabic Sign Language dataset. This produced the best result in the project, increasing the test accuracy to 95.19%. The fine-tuned MobileNetV2 model was selected as the final model for deployment.

VI. EXPERIMENTAL SETUP

All model training and evaluation experiments were conducted using Google Colab. The models were implemented using TensorFlow and Keras. The dataset was stored in Google Drive and loaded into the Colab environment. The training process used categorical cross-entropy loss and the Adam optimizer.

For the CNN experiments, the input image size was 64×64 pixels. For the MobileNetV2 experiments, the input image size was 224×224 pixels. The batch size was set to 32. Early stopping, model checkpointing, and learning rate scheduling were used during the stronger training stages to improve stability and preserve the best-performing model.

The following evaluation metrics were used:

- **Accuracy:** measures the percentage of correctly classified test images.
- **Loss:** measures prediction error during training and evaluation.
- **Precision:** measures how reliable each predicted class is.
- **Recall:** measures how well the model detects each actual class.
- **F1-score:** balances precision and recall.
- **Confusion matrix:** analyzes class-level errors.

Accuracy was computed as:

$$Accuracy = \frac{Correct\ Predictions}{Total\ Predictions}. \quad (1)$$

Precision, recall, and F1-score were used to provide a more detailed evaluation, especially because the dataset subset was moderately imbalanced. Accuracy alone can hide class-specific weaknesses; therefore, confusion matrix analysis and per-class performance were essential parts of the evaluation. Precision, recall, and F1-score are expressed as:

$$Precision = \frac{TP}{TP + FP}, \quad (2)$$

$$Recall = \frac{TP}{TP + FN}, \quad (3)$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}. \quad (4)$$

These metrics were useful because they show whether a class is being over-predicted or missed. For example, a class can have high recall but lower precision if the model predicts that class too often. Therefore, the evaluation focused on both global accuracy and class-level behavior.

VII. RESULTS AND DISCUSSION

The experimental results show a clear improvement across the different training stages. Table II summarizes the main experimental results. Fig. 3 visualizes the test accuracy comparison across all experiments.

The baseline CNN achieved 51.44% test accuracy. This result indicates that a simple CNN trained from scratch was able to learn some visual features from the dataset, but its performance was limited. After applying data augmentation, dropout, and longer training, the CNN accuracy improved to 69.23%. This confirms that augmentation and regularization helped the model generalize better.

The MobileNetV2 feature extraction model produced a much higher accuracy of 88.46%. This improvement demonstrates the value of transfer learning, especially when the dataset size is limited. MobileNetV2 had already learned

TABLE II: Experimental Results Comparison

Experiment	Image Size	Epochs	Test Loss	Test Accuracy
Baseline CNN	64×64	5	1.2822	51.44%
CNN + Augmentation + Dropout	64×64	10	1.1354	53.36%
CNN + Augmentation + Dropout	64×64	20	0.8850	65.38%
CNN + Augmentation + Dropout	64×64	40	0.7735	69.23%
MobileNetV2 Feature Extraction	224×224	20	0.3352	88.46%
Fine-tuned MobileNetV2	224×224	10	0.1197	95.19%

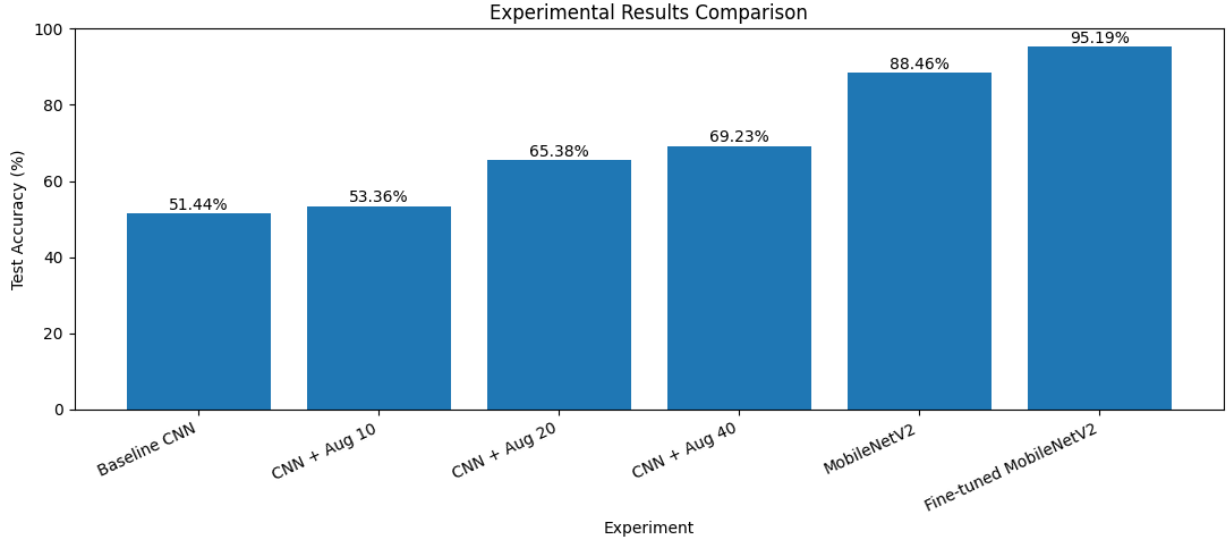


Fig. 3: Experimental results comparison across all trained models.

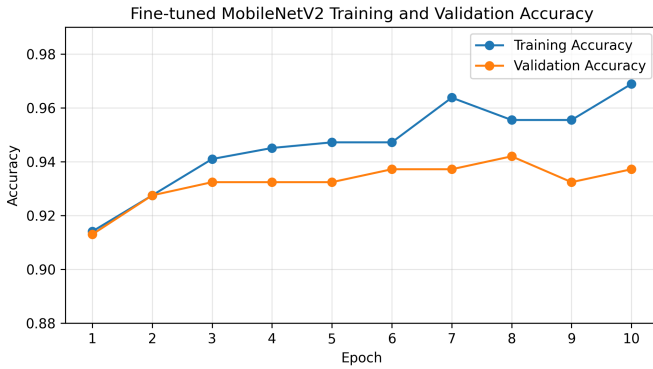


Fig. 4: Training and validation accuracy during MobileNetV2 fine-tuning.

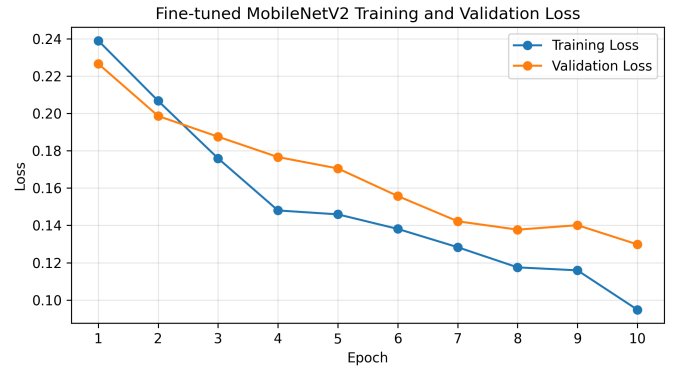


Fig. 5: Training and validation loss during MobileNetV2 fine-tuning.

useful visual features from large-scale image data, and these features were useful for extracting patterns from sign language images.

The best result was achieved after fine-tuning MobileNetV2. The final model reached 95.19% test accuracy with a test loss of 0.1197. This represents an improvement of 43.75 percentage points compared with the baseline CNN. The result shows that fine-tuning the final layers of MobileNetV2 allowed the model to better adapt to the selected Arabic Sign Language classes.

Fig. 4 and Fig. 5 show the fine-tuning training behavior.

The training accuracy increased gradually while validation accuracy remained high. The loss curves also decreased overall, indicating that the fine-tuned model learned more suitable features for the selected signs. The gap between training and validation performance was not severe, suggesting that the final model did not show extreme overfitting under the selected training settings.

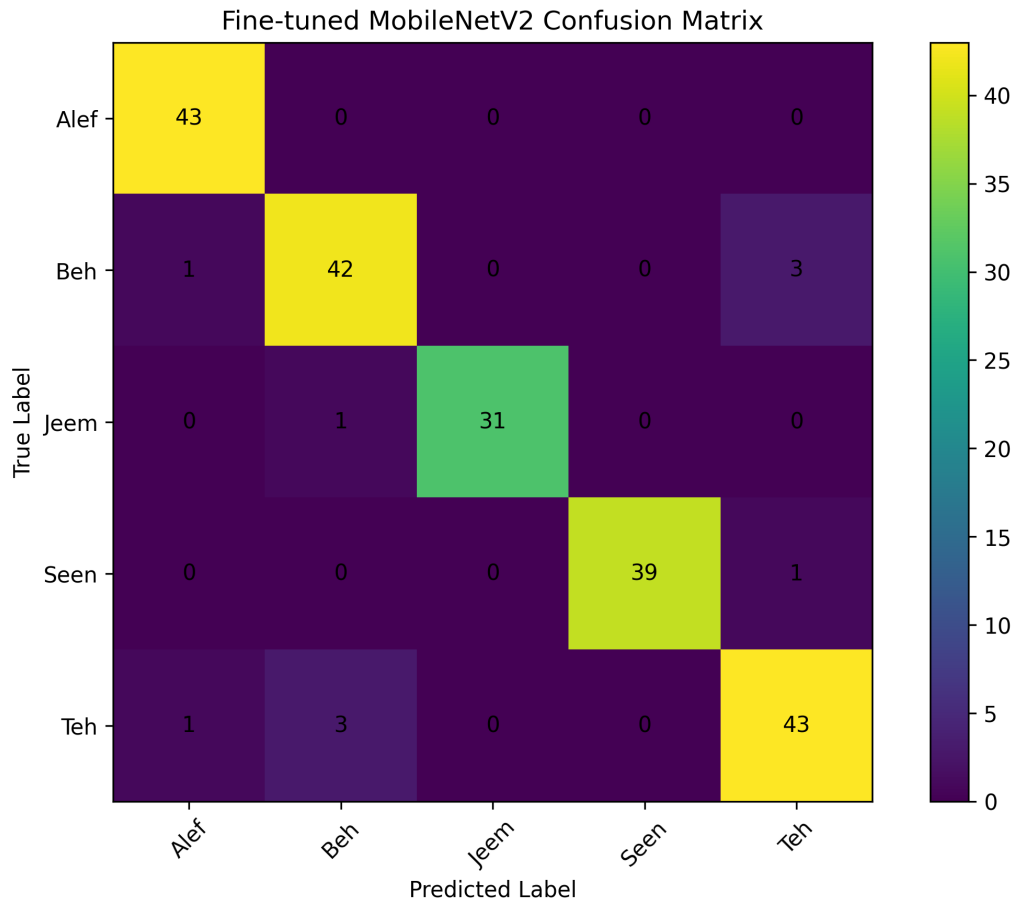


Fig. 6: Confusion matrix of the fine-tuned MobileNetV2 model.

VIII. CONFUSION MATRIX ANALYSIS

The confusion matrix for the fine-tuned MobileNetV2 model is shown in Fig. 6. The model correctly classified 198 images out of 208 test images, producing only 10 misclassified images.

The confusion matrix shows strong class-level performance. Alef was classified perfectly, with 43 correct predictions out of 43 test samples. Jeem was also classified with high accuracy, with 31 correct predictions out of 32 samples. Seen achieved 39 correct predictions out of 40 samples. Beh and Teh had slightly more errors compared with the other classes.

Most remaining errors occurred between visually similar signs. In particular, Beh and Teh were the most confused classes. The model predicted Teh for three Beh images and predicted Beh for three Teh images. This is expected because some hand shapes and finger positions can look visually similar depending on camera angle, distance, lighting, and hand orientation.

This analysis is important because it shows that the model's errors are not random. Instead, the errors are concentrated in visually similar classes. This helps guide future dataset improvement and model enhancement.

Table III summarizes the class-level performance derived

TABLE III: Class-level Performance of the Fine-tuned MobileNetV2 Model

Class	Precision	Recall	Correct/Total
Alef	95.56%	100.00%	43/43
Beh	91.30%	91.30%	42/46
Jeem	100.00%	96.88%	31/32
Seen	100.00%	97.50%	39/40
Teh	91.49%	91.49%	43/47

from the confusion matrix. The table confirms that Alef, Jeem, and Seen were recognized very strongly, while Beh and Teh remained the most challenging classes. This agrees with the visual inspection of the wrong predictions and supports the conclusion that additional training data for Beh and Teh would be the most useful next improvement.

IX. ERROR ANALYSIS AND DATASET IMPROVEMENT STRATEGY

After evaluating the final model, the misclassified test images were extracted and reviewed. Fig. 7 shows the misclassified images from the fine-tuned MobileNetV2 model. Each image includes its true label, predicted label, and confidence score.

The error analysis showed that the remaining mistakes were mainly related to three factors. First, some signs were visually similar, especially Beh and Teh. Second, some images contained challenging conditions such as different backgrounds, hand distance, or lighting variation. Third, some predictions had relatively high confidence even when incorrect, which indicates that the model learned strong but sometimes confusing visual patterns.

It is important to note that correctly labeled test images should not be removed only to increase accuracy. Removing difficult but valid test images would make the evaluation unfair. Instead, the correct approach is to use these errors to guide dataset improvement. If an image is corrupted, unclear, or incorrectly labeled, it can be corrected or removed during dataset cleaning. However, if the image is valid but difficult, it should remain in the test set and similar images should be added to the training set only.

The recommended improvement strategy is as follows. First, misclassified images should be manually reviewed and categorized by error type, such as similar sign, low quality, unusual angle, background issue, or possible label error. Second, more training images should be collected for the most confused classes, especially Beh and Teh. Third, realistic augmentation should be applied to represent real-world conditions such as brightness changes, small rotations, zoom, and hand position shifts. Finally, the model should be retrained and evaluated again using the same test set and, if possible, an additional real-world mobile test set.

This error analysis section is important for the defense of the project. It shows that the model was not evaluated only by a single accuracy number. Instead, the mistakes were extracted, inspected, and connected to a practical improvement plan. This is a more scientific approach than simply reporting the best accuracy value.

X. MOBILE APPLICATION INTEGRATION

After training and evaluating the final model, it was converted to TensorFlow Lite for mobile deployment. The TensorFlow Lite model was integrated into a Flutter mobile application called Tellisi App. The app was designed as a practical prototype to demonstrate how the trained AI model can be used outside the notebook environment.

Tellisi App supports two recognition modes. The first mode allows the user to upload an image from the device gallery. The selected image is resized to 224×224 pixels, preprocessed using the same MobileNetV2 preprocessing used during training, and passed to the TensorFlow Lite model. The app then displays the predicted Arabic sign letter, the confidence score, and the full probability distribution across all five classes.

The second mode supports live camera recognition. In this mode, the app captures frames from the camera stream and performs recognition at regular intervals. When the model predicts a sign with high confidence, the result is displayed using a green visual indicator. This mode demonstrates real-time interaction, although real-world performance may vary

depending on lighting, background, camera quality, and hand position.

The Flutter application was implemented using a DDD/Clean Architecture inspired structure. The presentation layer contains the user interface screens, including the home page, image upload recognition, live camera recognition, and about page. The domain layer defines prediction entities, repository interfaces, and use cases. The data layer handles TensorFlow Lite model loading, image preprocessing, inference execution, and label mapping. Cubit was used for state management to handle loading, prediction, success, and error states in a reactive and organized way. Fig. 8 summarizes the mobile application architecture.

Fig. 9 and Fig. 10 show several screens from the implemented mobile application. The home page provides navigation to the image upload and live camera modes. The About page presents the project title, student name, supervisor name, technical stack, and model performance summary. The image upload screen displays the selected image, the predicted class, confidence value, and the probability distribution across all classes. The live camera screen shows recognition results directly on the camera preview.

The mobile deployment demonstrates that the trained model can be moved from the experimental environment into an interactive application. This makes the project more practical and shows the connection between artificial intelligence model development and real-world software implementation.

XI. USER INTERFACE AND PROJECT BRANDING

The user interface was designed to support a clear classroom demonstration. The application uses Arabic text because the target context is Arabic Sign Language recognition. The main screen presents the project goal, the supported recognition modes, and a simple navigation structure. This avoids unnecessary complexity and helps the audience focus on the AI behavior.

The project was renamed Tellisi App to reflect the student's identity and ownership of the implementation. The logo shown in Fig. 11 was used inside the mobile app and in the About page. The application branding is not a replacement for the technical contribution; rather, it makes the prototype look complete and professional for presentation. A polished interface also helps demonstrate that the model was not only trained in Colab but actually deployed into a usable mobile software prototype.

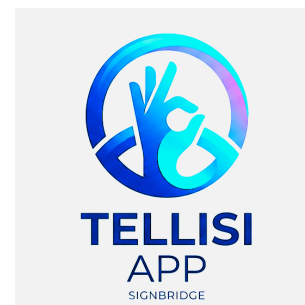


Fig. 11: Tellisi App logo used in the mobile prototype.

The About page documents the student name, supervisor name, technical stack, supported classes, and final model accuracy. This is useful for academic presentation because it makes the app self-explanatory. The app can be opened during discussion, and the evaluator can directly see that the system uses Flutter, Cubit, DDD/Clean Architecture, TensorFlow Lite, and a fine-tuned MobileNetV2 model.

XII. ON-DEVICE INFERENCE PIPELINE

The mobile inference pipeline was designed to match the training pipeline as closely as possible. This is important because a neural network expects the same type of input distribution during inference that it received during training. If the model is trained using MobileNetV2 preprocessing but the mobile application sends raw image values, the predictions can become unreliable even if the model performs well in Colab.

The on-device inference pipeline follows four main steps. First, the user provides an image either by selecting it from the gallery or by using the camera stream. Second, the image is decoded into RGB pixel values and resized to 224×224 pixels. Third, each pixel value is normalized using the MobileNetV2 preprocessing rule so that the input range becomes approximately $[-1, 1]$. Finally, the processed tensor is sent to the TensorFlow Lite interpreter, which returns five output probabilities corresponding to Alef, Beh, Jeem, Seen, and Teh.

The app displays both the highest-confidence prediction and the complete probability distribution. Showing all class probabilities is useful during testing because it reveals whether the model is strongly confident or confused between similar signs. For example, if Beh and Teh have close probabilities, this indicates visual ambiguity even if the top prediction is correct. This design decision also helps during the classroom discussion because it demonstrates that the model produces a probability vector rather than only a text label.

The live camera mode uses frame-based prediction at regular time intervals instead of predicting every frame continuously. This reduces computation and helps avoid unnecessary device load. The result is displayed using a confidence-based green indicator when the model is confident enough. This makes the interface clear for demonstration and helps the user quickly understand whether the sign was recognized.

XIII. REPRODUCIBILITY AND IMPLEMENTATION NOTES

Reproducibility was considered during the experimental workflow. The dataset was split using a fixed random seed and stratified sampling. The class folders were processed in a sorted order to reduce accidental changes in the train, validation, and test split. The final train, validation, and test dataframes were saved to disk so that later model evaluations could use the same split instead of creating a new random split.

Several files were preserved as final project artifacts. The best Keras model was saved as `extttbest_mobilenet_finetuned.keras`. The TensorFlow Lite model used by the Flutter application was exported from the fine-tuned Keras model. The experimental results table was

saved as a CSV file. The confusion matrix, wrong prediction grid, training curves, and app screenshots were also saved as image files for use in the report and presentation.

This organization is important because Colab runtime variables are temporary and disappear when the session is restarted. However, files stored in Google Drive remain available. Therefore, the project workflow separates temporary runtime objects, such as generators and model variables, from permanent artifacts, such as trained models, CSV results, and figures. This made it possible to continue the project after restarting Colab without repeating all training steps from the beginning.

XIV. THREATS TO VALIDITY

Although the experimental results are strong, several threats to validity should be considered. The first threat is dataset scope. The current system uses only five letters, so the reported accuracy should not be interpreted as performance on the full Arabic Sign Language alphabet. The second threat is dataset similarity. The train, validation, and test images come from the same dataset distribution. Real mobile camera images may contain different backgrounds, lighting, image blur, and hand distances.

A third threat is repeated experimentation. Multiple models were trained and compared, and the best result was selected. This is acceptable for development, but the final model should ideally be validated on an additional independent real-world test set. A fourth threat is that the model classifies full images without explicit hand localization. As a result, background features may influence predictions. A future hand detection stage could reduce this risk by cropping or focusing on the hand region before classification.

Despite these threats, the project remains valid as a semester-level applied AI project because the test set was separated from the training set, the same test set was used for the final evaluation, and the errors were analyzed rather than hidden. The limitations are also clearly stated and can guide future development.

XV. ACADEMIC INTEGRITY AND DEFENSE DISCUSSION

A common question in student AI projects is whether the model was built from scratch or whether a ready model was used. In this project, both approaches were used in a transparent way. The baseline CNN was built and trained from scratch. This model established an initial performance level of 51.44% test accuracy. The improved CNN experiments also trained custom CNN models with augmentation and dropout.

The final high-performing model used MobileNetV2, which is a pre-trained model. This does not mean the project is not an AI project. Instead, it means the project used transfer learning, which is a standard and accepted technique in computer vision. The MobileNetV2 base was first used as a feature extractor, and then fine-tuning was applied to adapt the model to Arabic Sign Language letter classification. Therefore, the project demonstrates both training from scratch and transfer learning-based improvement.

For discussion, the scientifically correct statement is that this work does not propose a new neural network architecture. Rather, it implements a full AI pipeline: dataset preparation, baseline training, model improvement, transfer learning, fine-tuning, evaluation, error analysis, TensorFlow Lite conversion, and Flutter deployment. This is a complete applied artificial intelligence workflow.

XVI. DETAILED DISCUSSION OF MODEL IMPROVEMENT

The improvement process in this project is important because it shows how model performance can be increased in a controlled and explainable manner. The first CNN experiment acted as a baseline. A baseline is necessary because it provides a reference point for measuring whether later methods are actually useful. Without a baseline, the final accuracy value would not clearly show the effect of each improvement step.

The baseline CNN produced 51.44% test accuracy. This result was not sufficient for deployment, but it confirmed that the dataset contained learnable visual patterns. The next experiments added data augmentation and dropout. Augmentation helped the model see different versions of the same image, while dropout reduced dependence on specific neurons. The improvement from 51.44% to 69.23% showed that the model benefited from regularization and longer training. However, the improvement also showed the limitations of training a small CNN from scratch on a limited dataset.

The largest improvement came from MobileNetV2 feature extraction. This result is expected because MobileNetV2 had already learned general image features from large-scale pre-training. Instead of learning edges, textures, and shapes from the Arabic Sign Language dataset alone, the model reused general visual knowledge and learned only a new classifier for the five selected classes. The test accuracy increased to 88.46%, showing that transfer learning was much more effective than the custom CNN for this dataset size.

Fine-tuning provided the final improvement. During feature extraction, the MobileNetV2 base was frozen and only the new classification head was trained. During fine-tuning, the final layers of MobileNetV2 were allowed to adjust using a small learning rate. This helped the model adapt high-level features to the specific visual patterns of Arabic hand signs. The result increased to 95.19% test accuracy. This improvement is meaningful because it reduced the number of wrong test predictions from 24 to 10 images.

The step-by-step improvement also makes the project easier to defend academically. Each stage has a clear purpose: the baseline demonstrates training from scratch, augmentation improves generalization, feature extraction applies transfer learning, and fine-tuning adapts the model to the target dataset. Therefore, the final result is not a random outcome; it is the result of a structured experimental process.

XVII. MODEL SELECTION RATIONALE

MobileNetV2 was selected for the final model for three main reasons. First, it provides strong feature extraction performance while remaining computationally efficient. This

TABLE IV: Model Development Stages and Purpose

Stage	Purpose
Baseline CNN	Establish initial performance using a model trained from scratch.
Augmented CNN	Improve generalization using data augmentation and dropout.
MobileNetV2 Feature Extraction	Reuse pre-trained visual features while training a new classifier.
MobileNetV2 Fine-tuning	Adapt final MobileNetV2 layers to the Arabic Sign Language dataset.
TFLite Deployment	Run the final model inside a Flutter mobile application.

is important because the final model was intended for mobile deployment, not only for notebook evaluation. Second, MobileNetV2 has a well-known architecture designed for resource-constrained devices. Third, it is available in Keras Applications, making it practical to use with TensorFlow and later convert to TensorFlow Lite.

Other models such as VGG, ResNet, EfficientNet, or Inception could also be considered for image classification. However, heavier models may require more memory and processing power, which can make mobile inference slower. Since Tellisi App is intended to run on a smartphone, a lightweight architecture was more suitable. MobileNetV2 provided a good balance between accuracy and deployability.

The final model was not chosen only because it produced the highest accuracy. It was also chosen because it was compatible with the deployment requirement. A model that achieves high accuracy in Colab but is too large or too slow for a mobile device would not satisfy the practical goal of the project. Therefore, the selected model had to satisfy both experimental performance and implementation feasibility.

XVIII. PRACTICAL DISCUSSION OF APP PERFORMANCE

The performance measured in Colab and the performance observed in the mobile application may differ. This is because the test set comes from the same dataset distribution, while mobile camera images may be captured under different conditions. In practice, the user's background, lighting, camera quality, hand distance, and image blur can all affect prediction quality. This is why a real-world mobile test set is recommended for future work.

The live camera mode is particularly challenging because frames may contain motion blur, partial hands, or backgrounds that were not present in the dataset. The model may also receive frames where the user has not completed the sign gesture yet. To reduce unstable behavior, the application does not need to run inference on every frame. Instead, it can analyze frames periodically and display results only when the confidence is high enough. This makes the user interface more stable and easier to interpret.

The image upload mode is more controlled than the live camera mode because the user selects a still image. It is also more suitable for demonstrating the full probability distribution because the image does not change continuously. The app displays all five probabilities, which helps identify uncertainty.

For example, if the model predicts Teh with 55% and Beh with 40%, the user can see that the model is confused. This is more informative than showing only the predicted label.

Adding pronunciation through text-to-speech also improves the application demonstration. When the user presses the pronunciation button, the app speaks the recognized Arabic letter. This turns the system from a pure classification demo into a more interactive assistive prototype. However, speech output is triggered by a button rather than automatically in the upload mode to avoid repeated or unexpected audio.

XIX. EDUCATIONAL VALUE OF THE PROJECT

This project has strong educational value because it connects several important areas of artificial intelligence and software engineering. From the AI side, it includes supervised learning, image classification, CNNs, data augmentation, transfer learning, fine-tuning, evaluation metrics, confusion matrix analysis, and error analysis. From the software side, it includes mobile deployment, TensorFlow Lite conversion, Flutter integration, DDD-inspired architecture, and Cubit state management.

The project also demonstrates the difference between a notebook experiment and a usable application. A high accuracy value in a notebook is not enough by itself. The model must be exported, loaded correctly in the application, and given input images with the same preprocessing used during training. The Flutter implementation therefore reinforces the concept of training-inference consistency.

Another important educational point is responsible reporting. The project does not claim that MobileNetV2 was invented or trained from zero. Instead, it clearly explains which model was trained from scratch and which model used transfer learning. This level of honesty is important in academic work and helps avoid misunderstanding during discussion. It also shows that using a pre-trained model is not a weakness when it is explained correctly and evaluated properly.

Finally, the error analysis encourages scientific thinking. Instead of hiding wrong predictions, the project extracts and studies them. This helps identify whether errors are caused by similar signs, poor image quality, unusual angles, or possible dataset issues. This approach is closer to real machine learning practice, where improving the dataset and understanding model failure cases are as important as training the model itself.

XX. LIMITATIONS

Although the final model achieved strong test accuracy, the project has several limitations. First, the current system recognizes only five Arabic Sign Language letters: Alef, Beh, Teh, Jeem, and Seen. It does not yet support the full Arabic alphabet. Second, the model is based on static image classification and does not explicitly model temporal hand movement. Third, the dataset images may differ from real-world mobile camera images, which can reduce performance in practical use. This issue is known as domain shift.

Another limitation is that the current model classifies the entire image and does not localize the hand region. Therefore,

backgrounds and irrelevant image regions may affect predictions. A future version could include hand detection or hand landmark extraction before classification. In addition, the live camera mode performs frame-based recognition rather than sequence-based recognition. This is acceptable for isolated letters but may not be sufficient for word-level or sentence-level sign language recognition.

XXI. FUTURE WORK

Future work can extend the project in several directions. First, the system should be expanded to include all Arabic Sign Language alphabet classes. Second, additional real-world images should be collected using mobile devices to improve robustness under practical conditions. Third, the most confusing classes, especially Beh and Teh, should receive additional training samples with different lighting conditions, hand distances, and camera angles.

Future versions of the application could also integrate hand detection or landmark-based preprocessing. This would help focus the model on the hand area and reduce the effect of background noise. Another improvement would be model quantization to reduce the TensorFlow Lite model size and improve inference speed on mobile devices. Finally, the system could be extended from isolated letter recognition to word-level or sentence-level sign language recognition.

XXII. RECOMMENDED DEFENSE POINTS

For the project discussion, three points should be emphasized. First, the project includes a model trained from scratch: the baseline CNN. Second, the final model uses MobileNetV2 through transfer learning and fine-tuning, which is a valid and common machine learning approach. Third, the project did not stop at training accuracy; it included testing, confusion matrix analysis, wrong prediction extraction, and Flutter deployment.

If asked whether the project used a ready model, the correct answer is yes, for the final stage. The project used MobileNetV2 as a pre-trained backbone, but it trained a new classification head and then fine-tuned the final layers on the Arabic Sign Language dataset. The project also trained a baseline CNN from scratch for comparison. This makes the improvement process transparent and academically defensible.

If asked how wrong images are handled, the correct answer is that valid test images should not be removed only to increase accuracy. Instead, wrong predictions are extracted and reviewed. Images with wrong labels or corrupted content can be corrected or removed during dataset cleaning. Valid but difficult images should remain in the test set, while similar new images should be added to the training set. This is the correct scientific method because it improves the model without making the test set artificially easier.

Table V summarizes several likely discussion questions and the scientifically correct answers. These points were included to make the project easier to explain during the classroom presentation and to avoid overstating the contribution.

A strong defense of the project should emphasize that the work is not only a final accuracy value. The project

TABLE V: Expected Discussion Questions and Scientifically Correct Answers

Question	Suggested Answer
Did the project build AI from scratch?	Yes, the baseline CNN was built and trained from scratch. The final best model used MobileNetV2 transfer learning and fine-tuning, which is a valid AI method for limited datasets.
Was a ready model used?	Yes. MobileNetV2 was used as a pre-trained backbone. A new classification head was trained, and the final layers were fine-tuned on the Arabic Sign Language dataset.
Why not use only the custom CNN?	The custom CNN improved from 51.44% to 69.23%, but the dataset size was limited. Transfer learning provided stronger visual features and raised accuracy to 95.19%.
How were wrong predictions handled?	Wrong predictions were extracted and reviewed. Valid difficult test images were not removed. Instead, the analysis guides future data collection for confusing classes such as Beh and Teh.
Why can app performance differ from Colab?	The phone camera may produce different lighting, background, distance, and blur. This domain shift can reduce real-world performance compared with the dataset test set.

demonstrates an engineering and scientific process: define the problem, prepare the dataset, train a baseline, improve the model, compare experiments, analyze errors, deploy the best model, and identify limitations. This is the complete workflow expected from an applied artificial intelligence project.

XXIII. CONCLUSION

This paper presented Tellisi App, an Arabic Sign Language letter recognition system based on deep learning and Flutter-based mobile deployment. The project followed a complete artificial intelligence workflow, beginning with dataset preparation and a baseline CNN trained from scratch, followed by improved CNN experiments, MobileNetV2 transfer learning, and MobileNetV2 fine-tuning.

The baseline CNN achieved 51.44% test accuracy. The improved CNN with augmentation and dropout reached 69.23%. MobileNetV2 feature extraction improved the accuracy to 88.46%, and fine-tuning the final MobileNetV2 layers achieved the best test accuracy of 95.19%. Confusion matrix analysis showed that the model performed strongly across the selected classes, with most remaining errors occurring between visually similar signs such as Beh and Teh.

The final trained model was converted to TensorFlow Lite and integrated into a Flutter mobile application. The application supports both image upload recognition and live camera recognition. This demonstrates that the trained model can be deployed in a practical mobile environment. Overall, the project achieved its main objective by training, improving, evaluating, analyzing, and deploying an Arabic Sign Language recognition model.

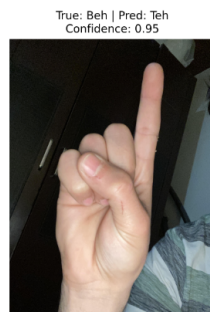
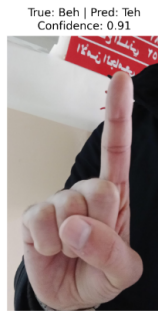
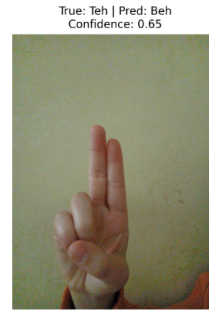
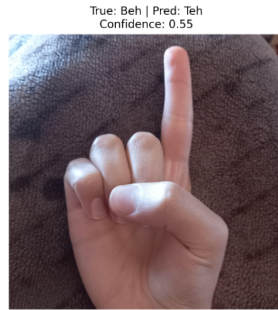
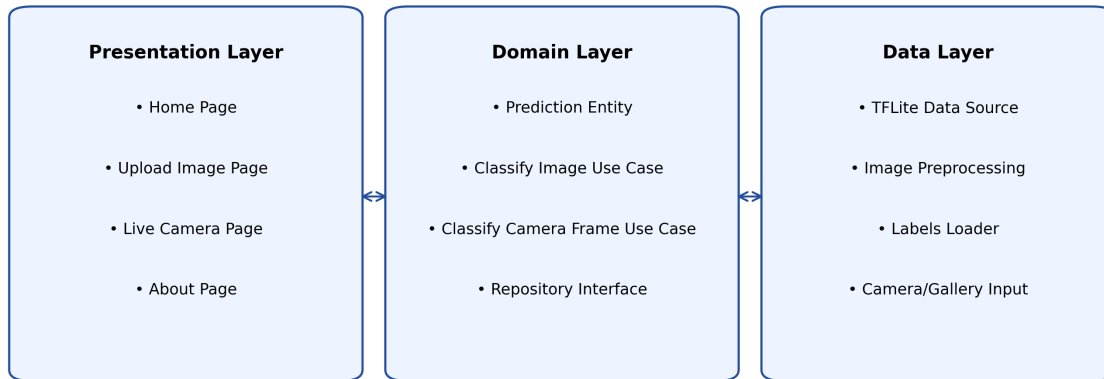


Fig. 7: Misclassified images produced by the fine-tuned MobileNetV2 model. The remaining errors mostly involve visually similar signs or challenging image conditions.



Tellisi App follows a DDD/Clean Architecture-inspired organization with Cubit state management.

Fig. 8: Flutter application architecture using a DDD/Clean Architecture-inspired organization and Cubit state management.

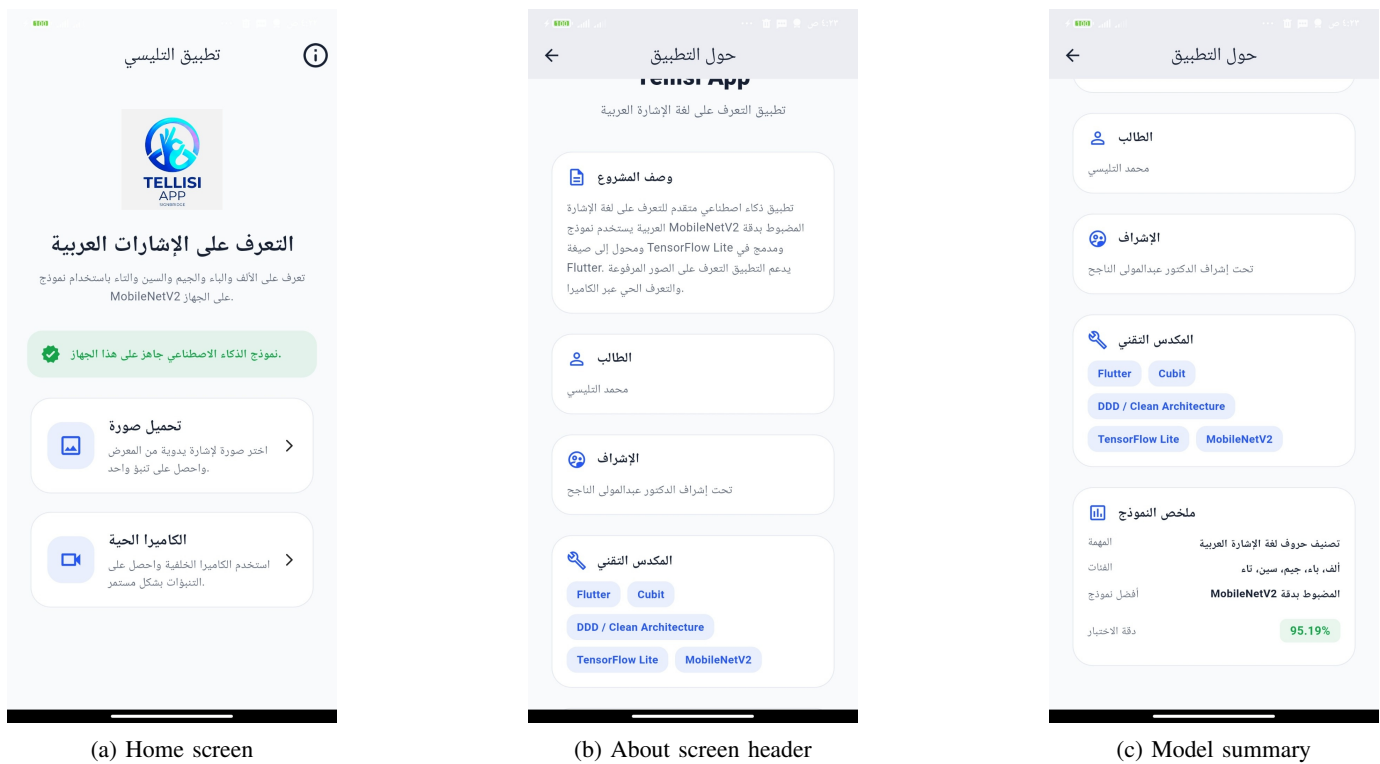
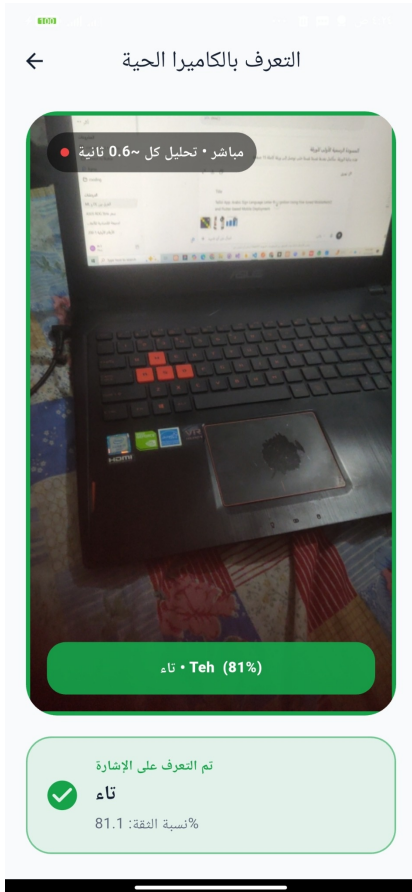
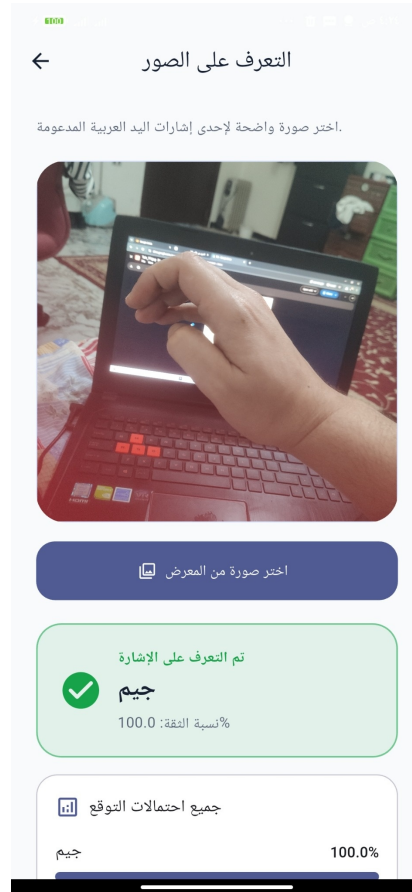


Fig. 9: Tellisi App overview screens showing the home page and About page.



(a) Live camera recognition



(b) Image upload result



(c) Class probabilities

Fig. 10: Tellisi App recognition modes: live camera inference, uploaded image recognition, and full probability output.

ACKNOWLEDGMENT

The author thanks Dr. Abdulmawla Al-Najeh for supervising this project. The author also acknowledges the creators of the RGB Arabic Alphabets Sign Language Dataset for making their dataset publicly available for research and educational use.

REFERENCES

- [1] M. Al-Barham, A. Alsharkawi, M. Al-Yaman, M. Al-Fetyani, A. El-nagar, A. A. SaAleek, and M. Al-Odat, "RGB Arabic Alphabets Sign Language Dataset," *arXiv preprint arXiv:2301.11932*, 2023.
- [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
- [3] O. Russakovsky *et al.*, "ImageNet large scale visual recognition challenge," *International Journal of Computer Vision*, vol. 115, no. 3, pp. 211–252, 2015.
- [4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [6] F. Chollet, *Deep Learning with Python*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2021.
- [7] TensorFlow, "TensorFlow Lite: Convert TensorFlow models," TensorFlow Documentation. [Online]. Available: https://www.tensorflow.org/lite/models/convert/convert_models
- [8] Keras, "Keras Applications: MobileNetV2," Keras Documentation. [Online]. Available: <https://keras.io/api/applications/mobilenet/>
- [9] Flutter, "Flutter documentation," Google. [Online]. Available: <https://docs.flutter.dev/>
- [10] M. Abadi *et al.*, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: <https://www.tensorflow.org/>